M68KWB Documentation

Latest edit: May 28th, 2025

- WHAT IS IT?
  - Features
  - Production cycle
  - UI Quick Overview
  - Advanced debugging features?
    - Accessing registers
    - Logging
    - Breakpoints
    - Memory
    - The Toolbox
    - Debug Primitives
    - Paranoid mode
    - Locks
    - Watchdog
    - MISC
  - Command Line
  - Copperlist (OCS only)
  - Emulation matrix
    - AMIGA
    - ATARI ST/STe
- WHY SHOULD YOU USE THIS?
  - If you're an Amiga Coder
  - If you're an Atari Coder
- THIS IS A WEB TOOL
  - You’re not a web expert? Not a problem
  - About File Paths
  - Tools needed to code and test
- BROWSING THE FX LIST
- CREATING YOUR OWN FX
- THE JAVASCRIPT / ASM PING-PONG
- LIMITATIONS & ANNOYING STUFF I NEED TO FIX
  - Assembler
  - Code and data sections
  - CPU Limitations
  - When shit hits the fan

THE AMIGA DISPLAY

WHAT IS IT?

M68KWB is an Amiga and Atari ST development tool that allows you to smoothly go from javascript prototyping to asm implementation. It's also useful if you only need to develop in pure assembly from scratch. I've been developing and using it for years now, and it really allowed me to code much faster, so I guess it's time to share now. If you need help / have questions:
email: soundydeadliners@gmail.com

Discord: soundy4264

Features

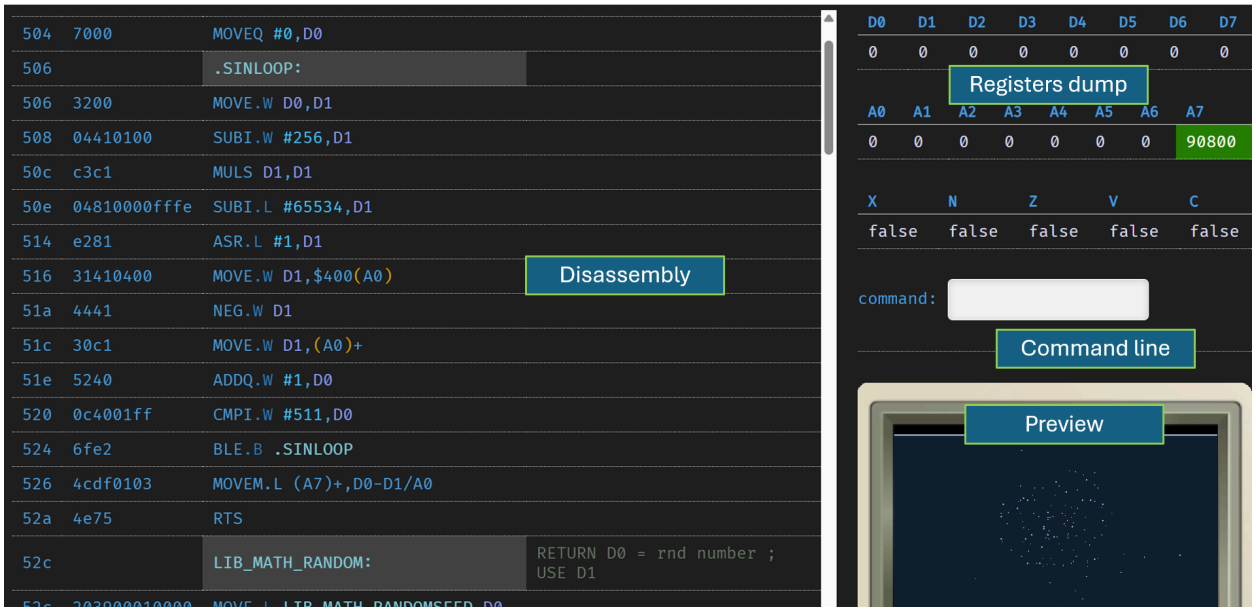
- **An assembler**, to assemble the source code you typed to actual opcodes in memory. The assembled instructions remain in RAM, you can't save them to disk or produce an executable for your final platform with it, so to produce your final demo's exe, you'll still need vasm (or any similar tool) in the end.
- **A disassembler**, because you may have used auto-modified code so the debugger must show you the proper instructions.
- **A debugger**, with classic and advanced features to trace your code, trigger breakpoints, etc.
- **Rudimentary emulation** of target platforms (Amiga OCS, Atari ST/STe for the moment), providing bitplane emulation, and a few extra hardware features. Note this tool is meant to make your m68k coding easier only. It does not emulate CPU cycles, DMA cycles, nor is it a full emulator for the Amiga or the ST.

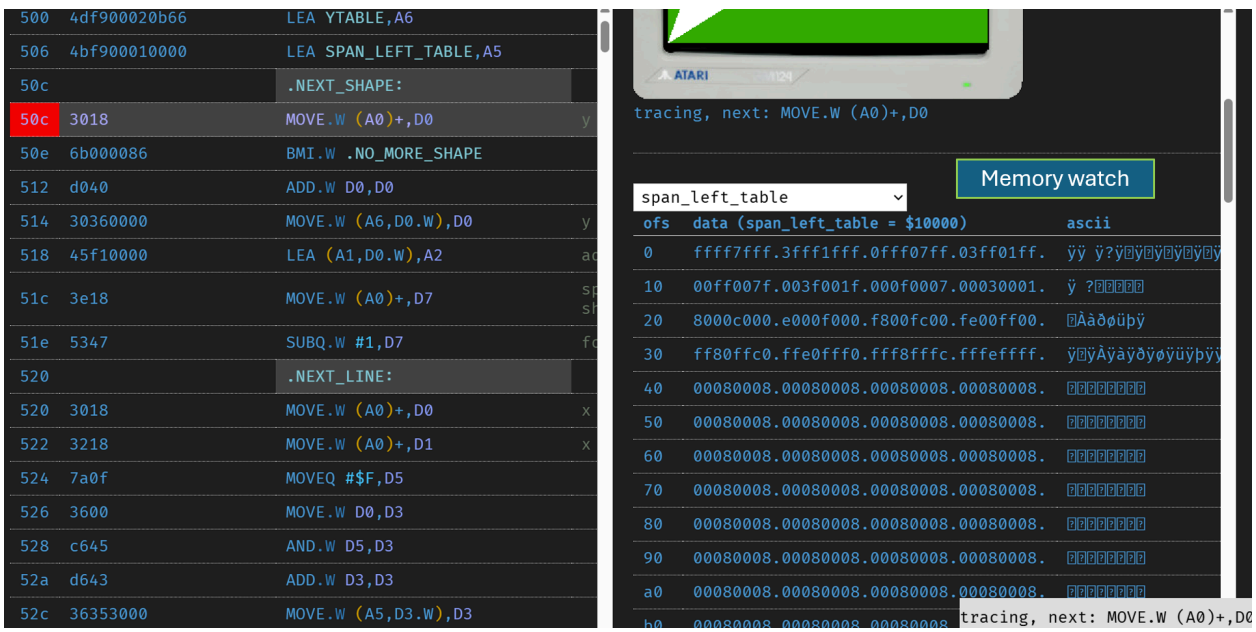
Production cycle

1. **Write your code**. You need a code editor for this. I highly recommend Visual Studio Code, and any m68k syntax extension so that you have syntax coloring when typing your code.
2. **Test & Debug**. Open M68KWB on your favorite browser, see how it works, debug, etc.
3. **Test & Debug**. As explained above, M68KWB does not fully emulate your target machine. Use your favorite assembler (vasm, Devpac, …) to generate an executable and test it with your favorite emulator (WinUAE, Hatari, …). If you're targeting Amiga, you can use Bartman's VS Code extension here too.

When prototyping your FX, you will spend a lot of time iterating between steps 1 & 2. Obviously, you don't need to check on the actual emulator as long as your FX does not work ;) Also, you may come back from step 3 to step 1

UI Quick Overview





Advanced debugging features?

Memory protection, code breakpoints, hardware breakpoints, value breakpoints, math overflow detection, but the most important is the ability to write and execute javascript in the middle of your asm code (called ***inline JS***). For example, imagine you have a table offset in `d0`, and you want to make sure you don't fetch outside of your table:

```
add.w d0,d0
;>JS if (d0.w > 1023) breakpoint("texture index overflow: " + d0.w);
;>JS if (d0.w < 0) breakpoint("texture index underflow: " + d0.w);
move.w (a0,d0.w),d0
```

This is a very simple example, but much more complex things can be achieved. Just note from the above:

- Any line starting with "`;>JS`" is considered as ***inline* JS**. If you assemble this with vasm, devpac or asmOne for example, it will be considered as a comment as it starts with '`;`'. So you can have as many as you want, it won't affect your final product (a bit like an assert in C).
- There's plenty of helpers to access asm labels, registers, and memory from inline JS (such as the `d0.w` above). Debugging primitives (draw points, lines, etc.) are also available. There's no limit to what you can do, you may call very complex JS functions featuring thousands of lines right in between 2 asm lines.

Here's a list of the available helpers from *inline* JS:

Accessing registers

Registers are stored by M68KWB in 2 Javascript `Uint32Array(8)`: `regs.a`, and `regs.d`. For example, the 2 below instructions do exactly the same thing:

```
;>JS regs.a[2] = 0;
move.l #0,a2      ; same as above
```

```
;>JS regs.d[4] = 42;
move.l #42,d4     ; same as above
```

Now, it can be cumbersome to use this to read values and log them. For example, if you want to log the value of the lower 16 bits of d0, you need to write:

```
;>JS print(regs.d[0] & 0xffff)
```

Actually, there's a series of helpers to read (and only read) registers values more easily from *inline* JS:

`<type><index>.<signed><part>`

type:	`d or a`
signed:	`i` (for signed), `u` (for unsigned). Defaults to signed if omitted
part:	`b, w, l or h`

examples:

`d0.ub`	unsigned 8 lower bits of d0
`d5.il`	signed 32 bits of d5
`d4.uw`	unsigned 16 lower bits of d4
`a2.w`	signed lower 16 bits of a2
`d1.ih`	signed higher 16 bits of d1

Logging

- Print a line in your browser's debugger (similar to console.log):

  ```
  ;>JS print("bad value d0 = " + d0.w);
  ```

- Show a "debug log" floating window at the top left of the screen with some debug info

  ```
  ;>JS ShowDebugLog("init done");
  ```

  ```
  ; >JS ShowDebugLog("a0: " + a0.l);
  ```

  ```
  ; >JS ShowDebugLog("offset 1:" + d0.w + "<br>offset 2: " + d1.w); //
  classic HTML tags work (here <br>) in the debug log window.
  ```

  ```
  ; >JS HideDebugLog(); // hide the debug log window
  ```

Breakpoints

```
;>JS if (d0.uw > 50000) debug("overflow"); // trigger a breakpoint
```

```
;>JS if (d0.uw > 50000) breakpoint("overflow"); // same as above
```

Memory

Memory is a huge Javascript `Uint8Array` called `MACHINE.ram[].` You may access this array from JS, but helpers are here for you:

Reading from a given address: getram(<adrs>, <bytes>(1,2 or 4), <signed> (true/false)

`;>JS getram(a0.ul, 2, true)` ⇒ returns the signed word at (a0)

Writing to a given address: setram(<value>,<adrs>, <bytes>(1,2 or 4))

`;>JS setram(42,a0.ul, 2)` ⇒ same as move.w #42,(a0)

Fetching labels: label(<name>)

```
BUFSIZE     EQU 1024
```

```
;>JS if(a5.ul-label("startBuffer") >= const("BUFSIZE")) debug("overflow");
```

Yep, const(<name>) fetches the value of a constant declared in asm using "`=`" or "`EQU`".

Fetching variables: variable(<label>, <size>, <signed>)

```
;>JS print("frame: " + variable("curFrame",2,false));
```

Now, imagine you need to fetch a variable within a struct:

```
      rsreset
var1        rs.w  1
var2        rs.w  1
structLen   rs.b  0
…
myStruct:   ds.b  structLen
```

```
;>JS if(struct("myStruct", "var2", 2, false) > 12) debug("overflow");
```

The syntax being: struct("<label name>", "<variable name>", size (1,2 or4), signed (true/false))

Protecting memory:

```
; trigger a breakpoint if anyone fucks with my buffer

;>JS CPU_DBG_WRITE_FORBID_START = label("myBuffer");

;>JS CPU_DBG_WRITE_FORBID_END = label("myBuffer") + const("bufferSize");

…

; we can update the buffer again now

;>JS CPU_DBG_WRITE_FORBID_START = -1;

;>JS CPU_DBG_WRITE_FORBID_END = -1;


; I’m supposed to write only to the bitplanes in this routine

;>JS CPU_DBG_WRITE_ALLOW_START = label("buf");

;>JS CPU_DBG_WRITE_ALLOW_END = CPU_DBG_WRITE_ALLOW_START + 32000;

…

;>JS CPU_DBG_WRITE_ALLOW_START = 0;

;>JS CPU_DBG_WRITE_ALLOW_END = 999999;

rts
```

Watching Memory:

There’s a memory watch at the bottom of the right pane. The UI allows you to set manually which address you want to watch, but it can be done in the code too, so that you can points the debugger to the right address automatically, which saves lots of time when debugging:

```
; >JS DBG_setDumpReg('a0');  ==> watch memory pointed by register a0

; >JS DBG_setDumpReg('myBuffer');  ==> watch memory pointed by label myBuffer
```

See also “**The Toolbox**” chapter below for more advanced ways to watch memory.

The Toolbox

Press `shift+t` to toggle it on/off. The Toolbox is a draggable window featuring several tools, one of them being the **watch**. If you tick the watch checkbox in the toolbox, any watches declared in your code will be shown. Here's how to declare a watch using *inline* JS:

```
;>JS WATCHES.add(<name>, <address>, <type>, <base>);
```

- `name` : the label that will appear in the watch
- `address` : the address to be watched
- `type` : `WATCH_BYTE`, `WATCH_WORD` or `WATCH_LONG`
- `base` : `WATCH_BIN, WATCH_DEC or WATCH_HEX`

Examples:

- Show/edit the value of a frame counter in hexadecimal:

```
;>JS WATCHES.add("frame", label("frame"), WATCH_WORD, WATCH_HEX);

update:

      add.w #1,frame

frame:

      dc.w  0
```

- Show/edit the value of a "hardcoded" address in memory:

```
;>JS WATCHES.add("color0", 0xdff180, WATCH_WORD, WATCH_HEX); // Amiga

;>JS WATCHES.add("color0", 0xff8240, WATCH_WORD, WATCH_HEX); // Atari
```

Note that the address is a Javascript number (hence the "0x" instead of "$" for hex number). This also means that arithmetic operations are ok, e.g. to watch color8, you could type:

```
;>JS WATCHES.add("color8", 0xdff180+8*2, WATCH_WORD, WATCH_HEX); // Amiga

;>JS WATCHES.add("color8", 0xff18240+8*2, WATCH_WORD, WATCH_HEX); // Atari
```

Debug Primitives

```
;>JS DEBUGPRIM.addLine(d0.h, d0.w, d1.h, d1.w, 0xff00ff); // x1,y1,x2,y2, HTML
color

;>JS DEBUGPRIM.addPoint(d0.w, d1.w, 0xff00ff, 4); // x,y,HTML color,radius
```

Paranoid mode

This mode is actually the default mode ;) It constantly checks for reading/writing to wrong addresses, jumping outside of the code section, data breakpoints, etc. If you want to get funky with your code (self-modified code, code generated in data section, etc.), you can disable and re-enable paranoid mode your asm file as such:

```
;>JS DEBUGGER_paranoid = false
```

... do your nasty stuff here

```
;>JS DEBUGGER_paranoid = true
```

It's recommended to keep `DEBUGGER_paranoid` to `true` as much as possible to guarantee the debugger is behaving properly.

Note also that verifications can be disabled on a per line basis just by adding `M68KWB_NOERROR` anywhere in a the comment following the instruction (on the same line)

Locks

Sometimes you want to make sure you don't overwrite a register to store something while you are still using it for something else. here's what you need:

```
; >JS lock(<register>,<mask>,<reason>);
```

- **register**: name of the register (d0..d7/a0..a7), case insensitive.
- **mask**: bits to protect (out of the register's 32 bits). Expressed as a JS number. Defaults to 0xffffffff if not given.
- **reason**: optional string (between quotes) to give an extra content to the error message

e.g.:

```
; >JS lock('d0', 0xffff, 'texture table offset'); // make sure the low 16 bits of d0 are not modified by any instruction.
```

After every CPU instruction, M68KWB will check if the protected bits were modified and trigger a breakpoint if it's the case. You will unlock a register as follows:

```
; >JS unlock('d0');
```

Note you can't unlock a mask, the full register is unlocked (but you may relock another mask immediately).

Now, you sometimes want to update the value of a locked register without unlocking it, e.g. keep it locked but change the locked value, here's the procedure:

```
; >JS lock('d0', 0xffff, 'texture table offset');
…
move.w #12,d0     ; M68KWB_NOERROR I know what I'm doing…
```

Just add `M68KWB_NOERROR` anywhere in a the comment following the instruction (on the same line)

Watchdog

M68KWB counts how many instructions have been executed since the last CPU invoke (e.g. at each update) and triggers a breakpoint if a maximum is reached. This makes infinite loops easier to investigate. There are 2 ways to modify this max count:

- `Change the value of watchdog_maxInstr in CPU_CONFIG` in file `config.js`
- Change it in your code (better!) with something like:

```
;>JS CPU_CONFIG.watchdog_maxInstr = 0; // disable watchdog because I'm about to launch a very long precalc because I'm a cheater that stores a big table in RAM

… do my nasty precalc here

;>JS CPU_CONFIG.watchdog_maxInstr = 400000; // re-enable watchdog
```

MISC

- **Pausing / Changing execution speed**

`;>JS onNewRunningSpeed(<amount>);` changes the execution speed (0..100), so that you can slow down for parts you want to debug, and have more time to see what's happening. 100 is normal speed

```
;>JS onNewRunningSpeed(30); ==> run at 30% of the normal speed (70% slow down)
....
;>JS onNewRunningSpeed(100); ==> back to full speed
```

`;>JS SET_PAUSE(true)` : just call this at the end of your update, you'll have to press the `'p'` key to unpause.

- ### Multiple-line inline JS

You can't declare a JS variable on a line and use it further. you have to set it as a field of DBGVAR.

<u>The below will NOT work</u>:

```
;>JS let oldD0 = d0.l // save d0 in a temp local variable
…                     // mess with d0
;>JS d0.l = oldD0     // nope, oldD0 will be undefined
```

<u>Do this instead</u>:

```
;>JS DBGVAR.oldD0 = d0.l      // save d0 as a field of a global lvariable
…                             // mess with d0
;>JS d0.l = DBGVAR.oldD0      // yep, that works
```

Check `fx/amiga/Sphere/v4/sphv4.asm`, and look for "`DBGVAR.`", you'll see tons of usages.

How the fuck did I end up here?

Yeah, sometimes I break on a line and wonder what the fuck is happening. 2 tools for that:

1. **Where am I?**

   just press the ‘w’ key while tracing

Where am I?

```
Just executed: ADD.W D0,D1
file: fx/amiga/StarField/v2/strfldv2.asm, line 54
```

Values on Stack

instr	file/line
BSR PLOT	file: fx/amiga/StarField/v2/strfldv2.asm, line 144
drawStars	Invoked from JS

Last taken branches

ofs	instr	file/line
678	BSR PLOT	file: fx/amiga/StarField/v2/strfldv2.asm, line 144
5da	RTS	file: fx/amiga/StarField/v2/strfldv2.asm, line 76
66e	BSR PROJECT	file: fx/amiga/StarField/v2/strfldv2.asm, line 142
666	BLE.B .ZOK	file: fx/amiga/StarField/v2/strfldv2.asm, line 138
608	RTS	file: fx/amiga/StarField/v2/strfldv2.asm, line 99
654	BSR ROT	file: fx/amiga/StarField/v2/strfldv2.asm, line 129

OK

2. **Time Machine**

When reaching a breakpoint, you may trace back previous instructions. M68KWB stores the registers values and IP for the last 256 instructions (reverts writes to RAM also, except blitter writes). Use keys x&y to play with the time machine.

Command Line

Debugging commands can be typed in the "command" text field. Supported so far:

- Hardware breakpoint: same as WinUAE: `w <index> <address> <size>`

  * `index :` 0,1,2 or 3 (only 4 hardware breakpoints supported, look for `MAX_HW_BPT` in `config.js` to change this).

  * `address:` hex address of the HW breakpoint (do not prefix with '$', just type the hex address)

* `size:` hex size of the HW breakpoint (do not prefix with '$', just type the hex size)

for example, to break on the long word at address 100: `w 0 100 4`

- Value breakpoint: `v <address> <value> <size>`

* `address:` hex address of the Value breakpoint (do not prefix with '$', just type the hex address)

* `size:` 1, 2 or 4

for example, to break when the word at $100 equals $1234: `v 100 1234 2`

Note that only 1 value breakpoint is supported at a time

- Set memory dump address: `m <address>`

* `address: hex address to dump (do not prefix with '$', just type the hex address)`

- Change register value : just type what you need. examples:

* `a0 = 0 ==> set a0 to 0`

* `a0 = d0 ==> copies d0's value to a0`

* `a0 = d2*5 ==> copies 5 x d2's value to a0`

* `d0 = $ff ==> set d0 to 255`

* `d0 = %111 ==> set d0 to 7`

- `Change current instruction: j <offset>`

* `offset:` ofs of the dest line as it appears in the debugger's left column

Note: `ofs` can be considered as the address of an instruction, it’s the same thing.

Copperlist (OCS only)

Press shift+c

adrs	opcode	instruction
1279c	008e,4c81	DIWSTRT = $4c81
127a0	0090,00c1	DIWSTOP = $c1
127a4	0092,0038	DDFSTRT = $38
127a8	0094,00d0	DDFSTOP = $d0
127ac	00e0,0001	BPL1PTH = $1
127b0	00e2,0b7c	BPL1PTL = $b7c
127b4	0120,0000	SPR0PTH = $0
127b8	0122,0000	SPR0PTL = $0
127bc	0124,0000	SPR1PTH = $0
127c0	0126,0000	SPR1PTL = $0
127c4	0128,0000	SPR2PTH = $0
127c8	012a,0000	SPR2PTL = $0
127cc	012c,0000	SPR3PTH = $0
127d0	012e,0000	SPR3PTL = $0

Emulation matrix

Once again, M68KWB is not meant to be an emulator. Some basic emulation is provided to check if your m68k code is behaving properly. I only implemented the features I needed, so you might be surprised some basic things are missing. A lot of them would be very easy to implement actually, so don't hesitate to contact me if you need something.

AMIGA

CPU instructions	Yes
Cycles	No cycles emulated (CPU, DMA)
Bitplanes	• 1 to 5 bitplanes (no HAM, no OCS fake HAM) • Low resolution only
Sprites	Not yet
Sound	No
Copper	• MOVE & WAIT are ok • SKIP not implemented yet (super easy to implement though) • The Copper may tap into DIWSTRT/STOP, BPLxPT, BPLxMOD, BPLCON0/1, COLORx, the values will be reflected. • Despite the fact no cycles are emulated, the copper emulation still takes into account how many "pixels" it takes to execute a copper instruction depending on the number of bitplanes.
Blitter	• Blitter operations are 100% executed whenever BLTSIZE is written (kind of forced HOG). • Not all features tested yet, so far so good. The implementation was stolen from Ruper Hausberger's SAE, and butchered to fit M68KWB.
RAM	All RAM is considered as CHIP
VBL interrupt	Yes, through $6c. Reading VPOSR/VHPOSR will not work.
HBL interrupt	Nope, try using the Copper
Disk	No disk emulation, use `incbin` in your asm files if you need to load a file.

ATARI ST/STe

CPU instructions	Yes
CPU cycles	No cycles emulated (CPU, DMA)
Bitplanes	320x200, 16 colors only
Sound	Only STe digitized sound is (partially) available
HW scrolling	STe HW scrolling registers are supported (pix offset, line offset, low 8 bits of screen address)
Blitter	Yes. Note that the Blitter always behaves like in HOG mode. In other terms, whatever the HOG flag's value, the Blitter will finish its task before giving the bus back to the CPU.
VBL interrupt	Yes, through $70.w
HBL interrupt	Yes, through Timer B, in scan lines count mode. Remember though, this is not a computer emulator, and DMA cycles are not emulated. This means M68KWB assumes your HBL callback has time to execute during the HBL, and will fully execute it before drawing the next line.
Disk	No disk emulation, use `incbin` in your asm files if you need to load a file.

WHY SHOULD YOU USE THIS?

If you're an Amiga Coder

- This won't replace vasm: It does not generate a binary
- This won't replace Win/FS-UAE: It does not emulate the machine, especially not the timings. Some basic features (e.g. bitplanes) are emulated though.
- This won't replace some fabulous VS Code extensions such as Bartman's fantastic one

If you're an Atari Coder

- This won't replace vasm: It does not generate a binary
- This won't replace Hatari (or your favourite emulator): It does not emulate the machine, especially not the timings. Some basic features (e.g. bitplanes) are emulated though.

THEN WHY THE FUCK DOES M68KWB EXIST?

This thing was made to be the 1st step in your development process: Leverage the ease of use of JavaScript (JS) to prototype complex algorithms before you progressively convert them to asm. It also allows you to embed powerful bug detection features within your asm code. To be honest, even when I want to code something from scratch in asm, involving no JS at all, the iteration speed and debugging features of M68KWB are the best way to go.

THIS IS A WEB TOOL

You're not a web expert? Not a problem

You don't know anything about HTML, Javascript (JS), or CSS? Well, not a problem, really. What you really need to know is fuckin' 68K assembly ;) Though, having notions of JS, or C, C++, Java, etc. will make your life easier to use the full debugging capabilities of the system.

About File Paths

As usual in web projects, all file paths are relative to the folder where index.html sits. In other terms, "`M68KWorkbench/`" is the root folder. When registering your own FX in the system, you can redefine what the root folder is though, so that each of your FX can have its own root folder.

Tools needed to code and test

All you need to "run" the project is to open "index.html" on a server. My recommended configuration is:

- Install Visual Studio Code: https://code.visualstudio.com/
- Open the root folder (M68KWorkbench) in Visual studio Code
- Install a VS Code Extension that can Launch a development local Server, such as "Live Server" by Ritwick Dey (https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer). Use it to open "`index.html`".
- Install a VS Code Extension that supports M68K syntax to get your asm code look beautiful, such as m68k by Steven Tattersall (https://marketplace.visualstudio.com/items?itemName=steventattersall.m68k).

BROWSING THE FX LIST

When you open "`index.html`" for the first time, for example using “Live Server” (see above), you’re shown a box that allows you to pick the FX you want to run. Next time you’ll run M68KWB, it will skip that “select FX” box and directly run the latest FX you worked on. To select another FX than the one that is currently running, press `SHIFT+z`, it will bring the box back and allow you to select another FX.

Note that you can also select a FX by changing the URL, using ?fx=<classname> (so that for example you can have 1 shortcut by FX), for example:

```
http://127.0.0.1:5500/index.html?fx=FX_tut1
```

Also note that when you are stuck with a FX that keeps on starting (because its name is stored in your browser’s localStorage) and you want to start with the FX chooser window, you can use:

http://127.0.0.1:5500/index.html?fx=reset

CREATING YOUR OWN FX

- A FX is made of at least 1 javascript file. The easiest way is to duplicate a sample ;)
- Besides the class, you must call function `REGISTER_FX` at the top of the file, outside the class declaration:

```
REGISTER_FX(params)
```

Here's an example of a params struct:

```
{
classname:"FX_TestSound"      // name of the FX class
platform:"STE",               // "OCS", "ST" or "STE"
hasAudio:true,                // (optional, default = false)
clickToStart:true,            // (optional, default = false)
rootPath:"fx/atari",          // (optional, default = null)
source:"TestSound.asm",       // (optional, default = null)
startPaused:false,            // (optional, default = false)
zoom:150                      // optional, in range [100..200]
}
```

Below are some more explanations about the params:

- **classname**: Name of the JS class for this FX
- **platform**: Defines the (very partially) emulated target platform. Available options are: "OCS", "ST" and "STE".
- **hasAudio**: Enables the audio engine (STE only for the moment, only samples, not YM).
- **clickToStart**: Will display a "start" dialog box before the fx starts. Useful when sound is needed (as browsers do not allow sound if the user did not click something)
- **roootPath**: Root folder for you fx. `include & incbin` directives will be relative to this. If for some reason you want to load a file that is not in your rootPath, you may use `SYSROOT/` at the beginning of your file path (works like HOME/ in some systems)
- **source**: Path to the main asm source file, relative to the **rootPath** param. Of course, you may use `include` directives in your asm files to include other asm files.
- **startPaused**: Always nice to have time to place breakpoints before it's too late. Use the 'p' key to unpause.

- **zoom**: Influences the zoom slider if you want to see a bigger rendering for your fx (values in [100..200] range).

Now that `REGISTER_FX` is called, you can write your class. All FX must implement the following:

- **`FX_Init()`:** Called once to init the FX
- **`FX_Update()`:** Called every frame BEFORE the bitplanes are transferred to screen. Call your asm updates here.

The below methods are optional:

- **`FX_DrawDebug(_ctx)`:** Called every frame AFTER the bitplanes are transferred to screen. Call your Javascript debug code here (canvas context draws). The "_ctx" param is the canvas's context to draw to. The coordinates are your screen's coordinates.
- **FX_Onkey**(keyCode) : Called whenever a key is down. The param keyCode is the javascript keycode (see https://css-tricks.com/snippets/javascript/javascript-keycodes/) NOTE: Avoid calling ASM code (e.g. using "invoke68K") from FX_OnKey. The reason is that you are already in a keyboard event, and if you want to trace your asm code, other key events won't be processed (e.g. 't' to trace). Instead, set a flag in FX_OnKey and trigger your asm code from FX_Update when the flag is on.

You're almost done: Open `index.html` and include your new Javascript file, e.g:

```
<script src='fx/amiga/mysuperfx/mysuperfx.js'></script>
```

Congratulations, you're done! Now go code a demo!

THE JAVASCRIPT / ASM PING-PONG

As seen just above, you have to implement a few methods. There is no specific feature to help you debug your pure JS code. Just use your browser's debugger that features breakpoints, watches, call stack, etc. The web is full of tutorials teaching you to debug JS code.

The **very important distinction to understand** is that you have 2 different kinds of JS in your code:

- The "*regular*" JS, located in your JS class code, within a javascript file
- The "*inline*" JS, located in your asm code using "`;>JS`"

The *inline* JS features syntax helpers that are not available in *regular* JS. The reason is that M68KWB adds an extra pass when assembling the asm code. For example, it will transform "`d0.uw`" into "`regs.d[0]&0xffff`".

So remember, the syntax you can use for *inline* JS using "`;>JS`" (see [Advanced Debugging features?]) is a simplified syntax that is not available in your *regular* JS methods. You'll find below a few hints to communicate between the *regular* JS part of your code, and you asm code:

- **Invoke a JS method of your FX from ASM**: `>JS MYFX.method();` The instance of the current FX is accessible through the global variable "MYFX". So you may access any member or method using MYFX.whatever… Works both *inline* and *regular* JS, but obviously, in your *regular* JS, MYFX is the same as the JS "`this`" keyword.
- **Access registers from *regular* JS**: regs.d[] and regs.a[] described [above] still works in *regular* JS. BUT, all the `dn.x`, e.g. `d0.uw` does not work in *regular* JS. In other terms, you can only access registers as an array of unsigned 32 bits values when you are in *regular* JS. That being said, the TOOLS singleton provides helper functions (see below).
- **TOOLS**: TOOLS is a singleton that features many functions to make your regular JS / ASM communication smoother. Check file `src/misc/JS_ASM_Tools.js`. All functions are documented. Easier, open file `jsdoc/index.html` and find the "JS_ASM_Tools" section. Remember: every function is accessed through "TOOLS.function". For example: `t.frames = TOOLS.getLabelAdrs("precalc_frames");`

One last pitfall: Javascript stores signed numbers, but the m68k stores 2-complement unsigned representation. So when using registers to assign javascript variables, you will always get the unsigned representation, e.g. if d0 contains -1, which is 0xffffffff and you do "`let foo = regs.d[0];`" then your foo variable won't be equal to -1, but to positive 0xffffffff. This is why you have conversion functions in `JS_ASM_Tools.js.` Here you would need to type:

"`let foo = TOOLS.toInt32(regs.d[0]);`"

Unless of course you want to interpret d0 as an unsigned number.

Note that when Javascript writes to an unsigned variable, it will do the 2 complement transformation by itself. So writing a javascript variable to regs.d[0] does not need a conversion.

LIMITATIONS & ANNOYING STUFF I NEED TO FIX

M68KWB is far from being perfect. There are things that need to be fixed, I will do someday, especially if other people than myself start using it. For the time being, I need to spend some time coding demos too….

Assembler

- "`include`" and "`incbin`" directives are processed before IFD-like directives. So you can't conditionally include/incbin a file. In other terms, all "include"/"incbin" directives are processed.
- On very, very rare occasions (please let me know if you find a repro), the assembler will not be able to process lines where a label and an instruction are on the same line, e.g:

  "`update: moveq #0,d0`" ⇒ put the `moveq` on the next line.
- Macros inside of macros are not yet supported
- Do not declare labels inside a REPT/ENDR loop
- Don't omit size for `DC/DS/DCB/BLK`. Always add `.b, .w` or `.l`.

Code and data sections

- There is a single section for code, all the code is assembled to the first 64Kb of RAM (see `CPU_CODE_SECTION_BYTES` in `config.js` to change this).
- There is a single section for all the data (incbin, DC.x, DS.x), and it's located right after the code section.
- So far so good? Almost… ALL code goes to the code section and ALL data goes to the data section, which means that if you declare data (e.g. dc.w) in the middle of code, the data address will not be near the code: It will be stored in the data section, with the other data. So if you are a fan of PC-relative code, you may not be happy. PC-relative addressing can be used though, but pc-relative instructions will be assembled and executed as non-pc-relative. The parser will simply ignore '(pc)' when parsing arguments. In other terms: you can type pc-relative code, but it will be assembled as non pc-relative code, the behavior of your code will remain the same though. OK, where's the catch then? Well, the opcode will be different, especially its size, so it can lead to different offsets if you use advanced optimizations such as auto-modified code, or jump tables upon pc-relative code .
- The "`section`" keyword is supported, but has no real effect, it just behaves as "`even`" : it makes sure the following data is 2 bytes aligned

CPU Limitations

- 68000 only. 68010,20,30,60,.... instructions are not supported.
- Clock cycles are somehow counted (to trigger interrupts), but not simulated at all. Don't ever use M68KWorkbench to know if your effect will run fast or slow on an actual machine
- Absolute data addressing: Everything is assembled as absolute long data. e.g: "`move.l a0,$70.w`" will be assembled and executed as "`move.la0,$70.l`". In other words, you may write ".w" in your code, and it will assemble and execute, but the opcode in memory will use ".l"
- pc-relative addressing: will be assembled and executed as non-pc-relative. The parser simply ignores '(pc)' when parsing arguments.
- SR register is not emulated. All operations on the SR register are assembled as NOP. This means you can still use SR in your code (to keep compatibility with the actual machine), but there will be no effect in the emulator. If you trace the code where you put your "SR-using" instructions, you will see NOPs instead. This should not be a big problem as you may install/clear interrupt vectors anytime, it will not crash the emulator.
- The Condition Code Register (CCR) exists, you may read/write it, but it will have no effect, and don’t expect to read the current proper flags from it. Don’t worry though, all CCR-based instructions (Bcc, Scc, ...) will work. It's just that internally, every state flag has its own variable instead of being a bit in a SR variable. This avoids bit manipulation and makes the emulation faster. I could update the SR register for real, maybe I'll do it in the future if users (if one day I'm not the only user on earth) complain.
- TRAP has no effect in most cases (same as NOP). Only on Atari ST/STe, a few basic system calls are supported through TRAP: `ssbrk, physbase, logbase, getrez, setscreen, setpalette, setcolor, rawconio (only function 0xff to read keyboard),mxalloc, malloc`. See "function ST_Trap" in "src/atari/atariST.js" for more details. Invoking TRAP in any other context will have no effect (except an error message in the browser's log window) and the program will continue to run. If you're NOT coding for Atari ST/STe (e.g AMIGA), TRAP will do absolutely nothing.
- The following instructions are NOT supported (hopefully not necessary to prototype a FX): `ABCD, CHK, CMPM, LINK, NBCD, RESET, RTR, SBCD, STOP, TAS, TRAPV, UNLK`.

When shit hits the fan

- **VASM is complaining**

M68KWB's syntax is more permissive than vasm. On the ST (compiled in devpac mode), I met cases where vasm did not support multiple data or bss sections. I suggest you organize your code as such:

```
main.asm:
    section code
        include "source1.asm"
        include "source2.asm"
        ...
    section data
        include "all_incbin.asm"
        include "all_dc.asm"
        ...
    section bss
        include "all_bss.asm"
```

- **My browser is stuck**
    - Your browser crashes whenever you try to open its developer tools, or to trace the javascript code ⇒ Open M68KWB in a private/incognito window (just open a private window and paste the URL).
    - You will probably get more info/context opening your browser's debugger. Maybe the code execution is stuck somewhere? Maybe the console window is filled with error messages?
    - Try the good old "have you tried turning it off and on again?" ;) Seriously, close the tab, and retry.
    - Your fx is somehow stuck in an infinite loop, and everytime you launch your browser, that stupid system re-loads automatically your fx, which freezes everything again.

      ⇒ Try resetting the current fx to come back to the default menu using `?fx=reset`:

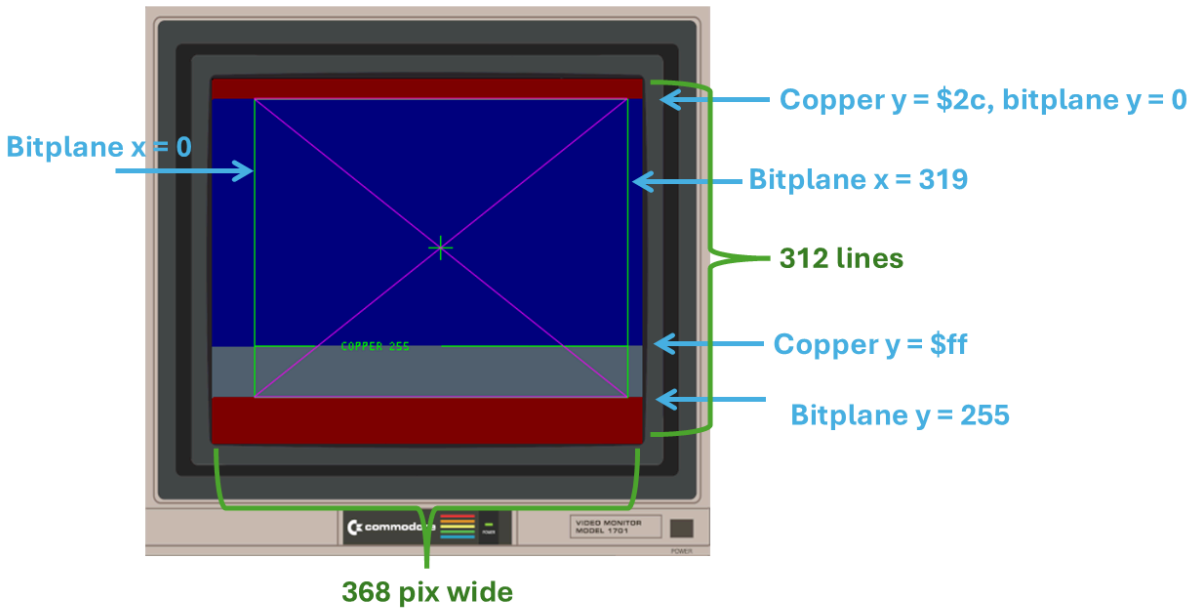
      `http://127.0.0.1:5500/index.html?fx=reset`

THE AMIGA DISPLAY

Only PAL low res is supported, hence:

- The maximum visible width is 368 pixels (http://amigadev.elowar.com/read/ADCD_2.1/Hardware_Manual_guide/node0085.html)
- The maximum visible height is 312 lines (http://amigadev.elowar.com/read/ADCD_2.1/Hardware_Manual_guide/node0063.html).

  ⇒ The monitor's image (`images/OCSMonitor.png`) is a 483x470 image that contains the monitor's frame and a 368x312 black square zone in the middle for the rendering.



The above image is generated by `fx/amiga/TestCopper`

OLD DOC ZONE...

Use the below info at your own risk! some is very probably deprecated, I just keep for myself to make sure I don’t forget anything in the “clean” doc.

```
/*
==============================================================================

                              PARSER SYNTAX & CONSTRAINTS


==============================================================================
*/



- CASE SENSITIVITY

--> It's not. Naming 2 different things by just changing the case won't work.
We are NOT case sensistive



- CONSTANTS

--> Expressions are supported, i.e. 'myConstant EQU (12+4)*3' is a supported
syntax



- LABELS

--> Local labels start with the '.' character

--> To keep compatibility with some assemblers, labels names should start at
the beginning of the line (no space or tab before)

--> Do not use reserved words for labels names (RS.B, RS.W, RS.L, SET, EQU, =,
MACRO, ENDM, REPT, ENDR, IFEQ, IFNE, IFD, IFND, ENDC, ENDIF, ...)



- RS.x
```

--> rs.x labels should NOT end with ':'

--> rs.x and their value must be on the same line, e.g. "myfield rs.b 2"

- INCLUDE and INCBIN are supported. Note though that they are processed before IFD-like directives. So you can't conditionally include or incbin a file. In other terms, all "include"/"incbin" directives are processed.

- REPT/ENDR

--> Do not declare labels inside a REPT/ENDR loop

Valid example:

```
font32UV:

UV      SET       12

        REPT      7

        dc.w      UV

UV      SET       UV+4

        ENDR
```

- CONDITIONAL ASSEMBLY

--> IFEQ <expression> Conditionally assemble the following lines if <expression> is zero.

--> IFNE <expression> Conditionally assemble the following lines if <expression> is non-zero.

--> IFD <symbol> Conditionally assemble the following lines if <symbol> is defined.

--> IFND <symbol> Conditionally assemble the following lines if <symbol> is undefined.

--> ENDC ends the directive

--> ENDIF ends the directive

--> ELSE reverses the condition of the directive

Limitations:

--> Does not seem to work properly when evaluating an expression defined in another file

--> Nested consitional assembly expressions do not work

- MACROS

<name> MACRO

Defines a macro which can be referenced by <name>. The macro definition is terminated by an ENDM directive.

When calling a macro you may pass up to 9 arguments, separated by comma. These arguments are referenced

within the macro context as \1 to \9. Parameter \0 is NOT valid. you may declare labels in macros by using \@ at the end of the label's name (see below).

example (I know, EXG instruction exists, it’s just an example):

```
    SWAPPTR MACRO

        move.l \1,d0

        move.l \2,d1

        move.l d0,\2

        move.l d1,\1
```

```
    ENDM

    ...

    SWAPPTR frontScreen,backScreen


WAIT_STE_BLITTER: macro

Loop\@:

       bset.b #7,BLT_MISC_1  ;test and set Busy-Bit

       nop                   ;do a NOP in any case

       bne.s Loop\@          ;if Busy-Bit was "1", go to Loop

endm
```

Note: You will get a "unknown argument type for: \n" error if you do not pass the right count of arguments to the macro

WARNING: MACROS INSIDE OF MACROS ARE NOT YET Supported

```
/*
==============================================================================

                                  INTERRUPTIONS


==============================================================================
*/
```

WARNING: VBL interrupt is evaluated every-time a CPU instruction is executed.

You NEED to call some routine in the update (even with just a couple of NOPs) so that some cycles are spent,

otherwise your VBL routine won't be invoked. In other words, if you just install your vbl callback at FX_Init and have no asm instructions executed at FX_Update, your vbl callback won't be called.

- ATARI ST/STe:

==========

==> VBL:

VBL interrupt is (partly) supported:

from: http://thethalionsource.w4f.eu/Artikel/Rasters.htm

Installation of a level 4 interrupt

```
instal4:
move.l    $70,old4+2    ;mark old vector
move.l    #new4,$70     ;set new vector
rts

new4:
;here are the seperate initialise routine
old4:
jmp       $000000   ;after old4+2 comes the old interrupt vector, so that the interrupt routines from the OS are still being executed
```

This will indeed install the routine pointed by a0 as a VBL interrupt. At runtime, the system counts cycles and triggers a VBL interrupt every M68K_TICKS_PER_FRAME cycles. M68K_TICKS_PER_FRAME is set to 8000000/50 on ST and 7140000/50 on OCS

The CPU will remain in interrupt mode until it reaches a RTE instruction. A different stack is used when inside an interruption.

Using XBIOS call #37 will not work. The $466 frame counter was implemented but never tested.

==> HBL:

HBL is not supported per se. Still, you can use Timer B in scan lines count mode.

note that this is for the moment the only Timer B supported feature:

```
    clr.b   $fffffa1b               ; disable timer b

    move.l  #timer_b,$120           ; move in my timer b address

    bset    #0, $fffffa07           ; turn on timer b in enable a

    bset    #0, $fffffa13           ; turn on timer b in mask a

    move.b  #1, $fffffa21           ; number of counts, every scan line

    move.b  #8, $fffffa1b           ; set timer b to event count mode
```

REMEMBER:

    This is not a computer emulator, and DMA cycles are not emulated.

    This means M68KWB assumes your HBL callback has time to execute during the HBL, and will fully execute it before drawing the next line.

- AMIGA:

Write to $6c to install your VBL routine. Reading VPOSR/VHPOSR will not work.

For HBL, use your copperlist

/*
==========

GENERATED & SELF MODIFIED CODE

==========
*/

Works, with a little extra constraint. An example can be found in: Tools/HTML/M68KWorkbench/fx/tutorials/tut4_automodified.asm

/*
==========

ASSEMBLY DEBUGGER

==========
*/

- Click on the left "ofs" column to toggle a breakpoint on/off. A red rectangle should appear.

Note that brakpoints are kept in your browser's localStorage, if you refresh your HTML page to relaunch your FX,

breakpoints will be restored

- Keys to navigate code: hit the 'h' key to show help

- Click on the value of a data register in the right pane to get a detailed view

```
==============================================================================
*/




/*
==============================================================================

                                    INVOKING CPU FROM JS

- Call a label in your loaded .asm file using 'invoke68K'. Example:
invoke68K("drawString");

- Access data registers using: regs.d[0], ... ,regs.d[7]

- Access address registers using: regs.a[0], ... , regs.a[7]


PARSER_getLabelAdrs(_label)

Equivalent to a LEA in asm: Fetches where a label is in RAM (as an index in
the MACHINE.ram array).

This function is useful to pass parameters to an ASM routine from Javascript.

@param      _label  :   the label to find

@return     the label's address (or NaN if error)

@throws     alert box if label is not found, or if no data follows the label
in the ASM source code(labels pointing to no data are not valid)


PARSER_getConstValue(_constName)

@return     the value of the constant in the asm file


MACHINE.setRAMValue(_value, _adrs, _size) // checks that _adrs can be written
to
```

```
MACHINE.getRAMValue(_adrs, _size, _signed)

MACHINE.ram[adrs] = 12  // directly writing the array


See also: misc/JS_ASM_Tools.js



==============================================================================
*/


/*
==============================================================================

                                INLINE DEBUGGING TOOLS

==============================================================================
*/


- See Tools/HTML/M68KWorkbench/src/misc/JS_ASM_Tools.js to see how to use
javacript arrays with asm

e.g.: function RAMToFile_bytes(_adrs, _size, _outputFileName)


; >JS showChunky(a0.l, PATH_W, PATH_H)
```

- Clear the current cycle counter:

; >JS M68K_CYCLES = 0; ==> NOTE: You HAVE to clear this counter yourself, nothing clears it automatically

- DEBUGGER_SetMemBrkPt(adrs, bytes) Triggers a breakpoint if a value is written to a specific address:

; >JS DEBUGGER_SetMemBrkPt(regs.a[0] + 96, 2); ==> watches the word located at 96(a0) and triggers a breakpoint if written to

- DEBUGGER_ClrMemBrkPt() clears any memory breakpoint set using DEBUGGER_SetMemBrkPt

; >JS DEBUGGER_ClrMemBrkPt();

==========

==========

CHAPTER 2 : ATARI ST FEATURES & LIMITATIONS

==========

```
==============================================================================

==============================================================================
*/


/*
==============================================================================

                                 MISC CONSTRAINTS

==============================================================================
*/


- For the moment, only 320x200 16 colors mode is supported

- ST and STE palettes are supported

- Only STE digitized sound is available (partially). No YM.

- Atari specific constants useable in your asm source files are listed in:
"src/atari/atariST_Constants.js".

- STe Hardware scrolling is supported

- STe Blitter is supported (some visual glitches still need to be fixed
though). Note that the Blitter always behaves like in HOG mode. In other
terms, whatever the HOG flag's value, the Blitter will finish its task before
giving the bus back to the CPU.


/*
==============================================================================


==============================================================================
```

==========

CHAPTER 3 : AMIGA FEATURES & LIMITATIONS

==========

==========

==========
*/

/*
==========

MISC CONSTRAINTS

==========
*/

- Amiga specific constants useable in your asm source files are listed in: "src/amiga/Amiga_Constants.js".

/*
==========

AMIGA RAM

RAM is a single continuous buffer. All RAM is considered as CHIP. There are 2 sections:

```
==============================================================================
*/
```

RAM is a big uin8 array (MACHINE.ram = new Uint8Array(MACHINE.ramSIZE + AMIGA_DATASIZE)). So addresses are actually offsets (indexes) inside this array.

All ASM instructions accessing the RAM will index in that array. For example, "move.w (a0),d0" will take the value of a0 an an index in the MACHINE.ram array.

In this framework, "address", "pointer", "index" or "offset" all refer to an index in the MACHINE.ram array.

Note that this is not the case for ROM addresses (especially custom registers addresses). In that case, the read/write CPU instructions check if the read/write address is a custom register or not before executing and tab into a different buffer.

MACHINE.allocRAM(_bytes, _allow=false, _align=4)

@param _bytes : bytes to allocate

@param _flags : ALLOW_READ, ALLOW_WRITE, ALLOW_READ|ALLOW_WRITE or null. If null, alllocated RAM is not automatically added to the allowed list

@param _align : allocated buffer alignment (4 by default)

@param _label : optional name for the memory block for debugging purposes

@return the index of the allocated buffer (index in the MACHINE.ram array)

@throws alert box if no more RAM

MACHINE.getRAMValue(_at, _size, _signed)

@param _at : 'address' to read (index in the MACHINE.ram array)

@param _size : bytes to read (1, 2 or 4).

@param _signed : true or false. Tells weather the read value should be signed or not

@return the value. Will return 0 if _size is not 1,2 or 4 (no exception or error thrown though)

MACHINE.setRAMValue(_at, _size, _signed)

@param _at : 'address' to write (index in the MACHINE.ram array)

@param _size : bytes to write (1, 2 or 4).

@return _at + _size. Will return just _at if _size is not 1,2 or 4 (no exception or error thrown though)

```
/*
==============================================================================
                                        ROM

==============================================================================
*/
```

ROM is implemented in customReg.js

Only custom registers are supported (missing some of them, especially the sound-related ones).

AMIGA_getCustom(_index)

Returns the current 16 bit value of a custom register

@param _index : custom register index (relative to $dff000)

@return the register's 16 bit value

AMIGA_getCustom_L(_index)

Returns the current 32 bit value of a custom register and the following register.

Useful to read a bitplane's current address for example, e.g; AMIGA_getCustom_L(BPL1PTH);

@param _index : custom register index (relative to $dff000)

@return the register's 32 bit value

AMIGA_getCustomFromPtr(_p)

Same as AMIGA_getCustom(), except that the parameter is not relative to $dff000, but an absolute address

AMIGA_getCustomFromPtr_L(_p)

Same as AMIGA_getCustom_L(), except that the parameter is not relative to $dff000, but an absolute address

AMIGA_setCustom(_index_, _value)

Sets the current 16 bit value of a custom register

@param _index : custom register index (relative to $dff000)

@param _value : value to set

AMIGA_setCustom_L(_index_, _value)

Sets the current 32 bit value of a custom register and the next one, e.g:

```
    let ptr = MACHINE.allocRAM(bplSize*_bitplanesCount);

    AMIGA_setCustom_L(BPL1PTH, ptr);
```

@param _index : custom register index (relative to $dff000)

@param _value : value to set

AMIGA_setCustomFromPtr(_p, _val)

Same as AMIGA_setCustom(), except that the parameter is not relative to $dff000, but an absolute address

AMIGA_setCustomFromPtr_L(_p, _val)

Same as AMIGA_setCustom_L(), except that the parameter is not relative to $dff000, but an absolute address

```
/*
==============================================================================

                                        BLITTER

==============================================================================
*/
```

- Same as a real Amiga for memory clear and block copy (including minterms). Just tap into BLT custom registers

- Line draw and blitter fill are not yet supported

```
/*
==============================================================================
```

AMIGA HELPERS

```
==============================================================================
*/
```

- Various helper functions are available depending on the target platform, e.g.:

```
    let copperlist = MACHINE.allocRAM(4096);

    regs.a[0] = copperlist;

    invoke68K("createDefaultCopperList");

    AMIGA_setCustom_L(COP1LCH,copperlist);


    AMIGA_drawChunky8(t.chunkyBuffer, t.CHUNKY_WIDTH, t.CHUNKY_HEIGHT, 320,
180);

    AMIGA_pix2Bitplane(_x,_y,_bpl);

    AMIGA_GetScreenHelper(_params);


/*
==============================================================================
```

ATARI HELPERS

```
==============================================================================
*/

    ST_drawPix(_x, _y, _colorIndex, _destScreen)

    ST_readPix(_x, _y, _formScreen)


/*
==============================================================================
```

VASM IS NOT HAPPY

```
==============================================================================
*/
```

Your code compiles on M68KWorkbench, but not vasm. The reason is that M68KWorkbench is a bit more permissive depending on your target machine:

- On ST, I met cases where vasm did not support multiple data or bss sections, you should organize your code as such:

```
main.asm:

    section code

        include "source1.asm"

        include "source2.asm"

        ...

    section data

        include "all_incbin.asm"

        include "all_dc.asm"

        ...

    section bss

        include "all_bss.asm"
```

- EQU directives must start at the beginning of the line (no space at the beginning of the line)

```
/*
==============================================================================
```

MY BROWSER IS STUCK

==========
*/

Yep, happens, that sucks. Just close the tab and relaunch. Try to set a breakpoint before is fucks up

/*
==========

LIBRARIES

==========
*/

- Memory Manager

    Provides malloc/free system with some debuggoing features.

    1. Define how many kilobytes of CHIP mem you want to be managed by the system (it's Amiga only, so 0 on Atari)

        LIB_MEM_CHIP_BLOCKS   = 100 ; for example here, 100 Kb. Note: must be a multiple of 4, max 32764

    2. Define how many kilobytes of default mem you want to be managed by the system (all platforms)

        LIB_MEM_BLOCKS = 500 ; LIB_MEM_CHIP_BLOCKS   = 100 ; must be a multiple of 4, max 32764

    chip mem and default mem blocks will be declared as bss_c and bss sections.

    3. Include lib_mem_api.asm : include "lib/lib_mem_api.asm"

    4. In our init, call lib_mem_init

So, to include the memory manager, here's how your code should look like:

```
LIB_MEM_CHIP_BLOCKS = 0     ; Atari example, no chip mem

LIB_MEM_BLOCKS    = 400   ; 400 kb bss section

include "lib/lib_mem_api.asm"

...

jsr lib_mem_init
```

Then, you want to allocate and free some memory.

```
    move.l          #10000,d0       ;   amount of bytes to be allocated

    moveq           #0,d1           ;   do not clear memory (use MEMF_CLEAR if
you want memory to be cleared)

    lea             freeMemList,a0  ;   linked list of all allocations (head
of list)

    jsr             lib_mem_alloc   ;   d0 will contain the allocated buffer
address

    ....

    lea             freeMemList,a0

    jsr             lib_mem_free

    ...

freeMemList:

    dc.l            0
```